Assignment-3

Sorting Algorithms

IMPLEMENTATION OF DIFFERENT SORTING ALGORITHMS

Institute Name- K.R. Mangalam University

Course Name: Data Structures

Program Name: B. Tech Computer Science and Engineering (AI and ML)

Faculty Name: Dr. Neetu Chauhan

BUILDING A SORTING SPA TO LEARN AND IMPLEMNENTS SORTING ALGORITHMS

By: Mehul Srivastava

Roll No.- 2501730149

1. Introduction:

Sorting is a fundamental operation in data structures used to arrange data in a specific order. Different sorting algorithms perform differently depending on the input data and their time complexity.

In this assignment, three sorting algorithms were implemented:

- Insertion Sort
- Merge Sort
- Quick Sort

We have also implemented Dataset generator in random, sorted and reverse order and noted their time using python time library

Their performance was analysed on different datasets to understand practical behaviour vs theoretical complexity.

2. Execution Table:

The following table shows the execution time (in milliseconds) for each sorting algorithms across different input sizes and types

Size	Type	Insertion	Merge	Quick
1000	Random	28.12	2.43	1.69
1000	Sorted	0.15	1.78	1.65
1000	Reverse	57.22	1.78	1.59
5000	Random	916.36	15.45	9.51
5000	Sorted	1.13	10.32	10.12
5000	Reverse	1540.83	10.01	9.56
10000	Random	3239.9	36.2	21.36
10000	Sorted	2.34	29.68	25.39
10000	Reverse	5939.38	21.99	21.71

3. Algorithms implemented:

3.1. Insertion Sort:

Insertion Sort works by picking one element at a time and inserting it into its correct position.

Time Complexity:

- Best Case: O(n)
- Worst Case: $O(n^2)$

Space: In-place

Stable: Yes

3.2. Merge Sort:

Merge Sort uses the divide-and-conquer approach by dividing the list and merging sorted halves.

Time Complexity: O(n log n)

Space: Uses extra memory/Out-place

Stable: Yes

3.3. Quick Sort:

Quick Sort selects a pivot and partitions the array around it.

Time Complexity:

- Average: O(n log n)
- Worst Case: $O(n^2)$

Space: In-place

Stable: No

4. Analysis for comparison between the different output time:

4.1. Random Data

Merge Sort and Quick Sort performed much faster than Insertion Sort. This matches theory because O(n log n) is better than $O(n^2)$.

4.2. Sorted Data

Insertion Sort performed very fast because it runs in O(n) in best case. Quick Sort became slower due to pivot behaviour.

4.3. Reverse Sorted Data

Insertion Sort performed worst because maximum shifting is required. Quick Sort also slowed down compared to random input. Merge Sort remains relatively stable

5. Complexity Analysis:

Algorithm	Time Complexity (average case)	Performance
Insertion Sort	$O(n^2)$	Slow for large inputs
Merge Sort	O(n log n)	Consistent
Quick Sort	O(n log n)	Fast but unstable

***Special Note on Quick sort:**

Quick Sort can degrade to $O(n^2)$ if pivot selection is poor. To avoid this, **random pivot selection** was used, which improves performance and avoids deep recursion.

6. Conclusion:

Merge Sort is the most reliable algorithm. Quick Sort is fastest on average. Insertion Sort is efficient only for small or nearly sorted data. Thus, the choice of sorting algorithm depends on the input type and size.